

Platforma cu trei aplicatii

Exercitiu · o singura poarta nginx in fata a trei aplicatii cu frontend si backend

Ce primesti: un `compose-platforma.yaml` cu opt containere si trei frontend-uri gata scrise. Backend-urile sunt imagini deja publicate, deci nu ai nimic de construit.

Ce ai de facut: un singur fisier, `conf/platforma.conf`, si apoi deploy-ul pe server.

Cat dureaza: conf-ul, o ora. Deploy-ul, inca una.

1. Ce e in platforma

Aplicatie	Frontend	Backend	Portul backendului
Scoala	<code>web-scoala</code>	<code>api-scoala</code>	8081
Notite	<code>web-notite</code>	<code>api-notite</code>	8080
Masini	<code>web-masini</code>	<code>api-masini</code>	8080

Mai e un container `mysql`, folosit de scoala si de notite, si containerul `proxy`, care esti tu.

SINGURUL PORT PUBLICAT E 80, AL PROXY-ULUI

Niciun frontend si niciun backend nu are `ports:` in compose. Pana nu scrii conf-ul, nu vezi absolut nimic in browser. Nu ai cum sa ocolesti nginx-ul intrand direct pe un port, cum faceai la platforma demo.

2. Cele trei backend-uri isi cer prefixul altfel

Asta e miezul exercitiului. Aceeasi informatie, trei forme:

Backend	Unde asculta el	Exemplu de ruta reala
<code>api-scoala</code>	de la radacina	<code>/courses</code> , <code>/auth/login</code>
<code>api-notite</code>	sub <code>/api/v1</code>	<code>/api/v1/notes</code>
<code>api-masini</code>	sub <code>/api/external</code>	<code>/api/external/cars</code>

Frontend-urile nu stiu nimic despre asta. Toate trei cer de la o adresa cu aceeasi forma: `/api/scoala/`, `/api/notite/`, `/api/masini/`. Tu le potrivesti.

3. Rutele pe care trebuie sa le faci

Ce cere browserul	Unde trebuie sa ajunga
<code>/</code>	redirectare catre <code>/scoala/</code>
<code>/scoala/</code>	frontendul scolii
<code>/notite/</code>	frontendul de notite
<code>/masini/</code>	frontendul de masini
<code>/api/scoala/courses</code>	<code>api-scoala</code> , ruta <code>/courses</code>
<code>/api/notite/notes</code>	<code>api-notite</code> , ruta <code>/api/v1/notes</code>
<code>/api/masini/cars</code>	<code>api-masini</code> , ruta <code>/api/external/cars</code>

4. Gata cand

Fiecare punct se verifica in browser, nu pe hartie.

a) Intri pe adresa serverului, fara nimic dupa ea, si ajungi la scoala.

b) La **Scoala**: apesi „Creeaza cont”, apoi „Autentificare”, iar bara de sus devine verde. Apesi „Incarca cursurile” si vezi un tabel cu cursuri.

c) La **Notite**: creezi cont, te autentifici, scrii un titlu si apesi „Adauga”. Notita apare in tabelul de dedesubt. Dai refresh, apesi „Incarca”, notita e tot acolo.

d) La **Masini**: apesi „Incarca masinile” si bara verde iti spune cate ai primit. Nu ai nevoie de cont aici.

e) Cele trei linkuri din meniul de sus te plimba intre aplicatii, si fiecare pagina arata cu stilurile ei, nu ca text simplu.

f) Scrii adresa cu `/scoala`, fara slash la final, si tot ajungi la scoala, intreaga.

5. Trei lucruri care te vor incurca

PAGINA APARE, DAR FARA CULORI SI BUTOANELE NU FAC NIMIC

Inseamna ca `style.css` si `app.js` au dat 404, desi `index.html` a venit. Paginile le cer cu cale relativa. Uita-te la ce ajunge de fapt la containerul din spate.

FARA SLASH LA FINAL, TOTUL SE RUPE

Aceiasi poveste ca la platforma demo, dar acum se vede in pagina, nu intr-un cod de raspuns. Un singur caracter e diferenta intre 200 si 404.

CONEXIUNE REFUZATA PE PORTUL 80

Nu e firewall si nu e containerul picat. Folderul `conf/` se monteaza intreg in nginx. Daca e gol, nginx porneste fara niciun bloc `server` si nu asculta nicaieri.

6. Local: porneste-o si joaca-te cu ea

Inainte de server, tine platforma pe calculatorul tau si strica-o intentionat. Asa inveti ce inseamna fiecare raspuns pe care il vei vedea mai tarziu in productie.

Pornire si oprire

```
docker compose -f compose-platforma.yaml config -q
docker compose -f compose-platforma.yaml up -d
docker compose -f compose-platforma.yaml ps
```

Prima comanda nu porneste nimic, doar verifica fisierul. Ia-ti obiceiul sa o dai mereu inainte de `up`.

Prima pornire dureaza un minut sau doua: `mysql` trebuie sa fie sanatos inainte sa porneasca scoala si notitele. In `ps` te uiti la coloana de stare, nu doar la faptul ca apar in lista.

```
docker compose -f compose-platforma.yaml down
docker compose -f compose-platforma.yaml down -v
```

ATENTIE LA -v

`down` opreste containerele si pastreaza baza de date. `down -v` sterge si volumul, deci pierzi conturile create si notitele scrise. Daca a doua zi „nu mai merge parola”, asta s-a intamplat.

Ce se intampla inaintur

```
docker compose -f compose-platforma.yaml logs api-scoala
docker compose -f compose-platforma.yaml logs -f --tail=50 api-notite
```

Al doilea ramane agatat si scrie pe masura ce apar linii noi. Lasa-l deschis intr-un terminal si apasa butoanele din pagina in altul: vezi cererile tale sosind.

```
docker compose -f compose-platforma.yaml logs -f proxy
```

Asta e jurnalul nginx-ului tau. Fiecare linie iti arata calea ceruta si codul cu care ai raspuns. Cand ceva nu merge, aici te uiti primul, nu in browser.

Intra in containere

```
docker compose -f compose-platforma.yaml exec proxy nginx -t
docker compose -f compose-platforma.yaml exec proxy nginx -s reload
docker compose -f compose-platforma.yaml exec proxy sh
```

Dupa orice modificare in conf: intai `nginx -t`, apoi `reload`. Nu opri si nu reporni containerul pentru o schimbare de configurare.

Cu `sh` intri in container. De acolo poti vedea reseaua din interior, asa cum o vede nginx:

```
wget -q0- http://api-masini:8080/api/external/cars
wget -S -q0- http://web-scoala:80/
```

DE CE CONTEAZA

Numele `api-masini` si `web-scoala` exista doar inaintur retelei create de compose. De pe calculatorul tau nu le poti apela. Exact acele nume le scrii in `proxy_pass`.

Experimente: strica ceva si uita-te in pagina

1) Opreste un backend si reincarca aplicatia lui:

```
docker compose -f compose-platforma.yaml stop api-scoala
```

Frontendul se incarca in continuare, dar butoanele nu mai aduc nimic. Uita-te in jurnalul proxy-ului ce cod apare. Porneste-l la loc cu `start` si reincearca.

2) Opreste un **frontend**, nu backendul:

```
docker compose -f compose-platforma.yaml stop web-notite
```

Ce vezi acum la `/notite/`, si prin ce se deosebeste de ce ai vazut la punctul 1? Celelalte doua aplicatii trebuie sa mearga mai departe.

3) Scoate slash-ul de la final dintr-un singur `proxy_pass` care trimite spre un frontend, da `reload` si reincarca pagina. Pune-l la loc dupa ce ai vazut ce se intampla.

4) Muta o aplicatie pe alta adresa: fa ca notitele sa raspunda la `/note/` in loc de `/notite/`, fara sa atingi niciun fisier din `web-notite/`. Daca trebuie sa modifci frontendul, inseamna ca ai mutat gresit.

5) Golesteti fisierul de configurare, da `reload`, si vezi cu ce te intampina portul 80. Apoi pune conf-ul la loc.

Cand modifici un frontend

Folderele `web-*` sunt montate in containere, nu copiate. Schimbi un fisier, dai refresh in browser si vezi modificarea. Nu e nevoie de `build`, nici de `restart`. Daca nu se schimba nimic, e memoria browserului: reincarca fortat.

7. Deploy pe server

Aceiasi pasi ca in `notite/ETAPE-NGINX.pdf`, cu o singura diferenta: aici nginx-ul e in container, deci nu instalezi nimic cu `apt` si nu atingi `sites-available`.

1. Deschizi portul 80 in Security Group.
2. Copiezi pe server folderul intreg, cu `compose-platforma.yaml`, cele trei foldere `web-*` si `conf/` cu fisierul tau in el.
3. Pornesti cu `docker compose up -d` si verifici cu `ps`.
4. Intri din browser pe IP-ul serverului si treci prin toate punctele de la capitolul 4.

CAND E TERMINAT

Cineva care primeste doar adresa serverului poate folosi toate trei aplicatiile, fara sa stie ca in spate sunt opt containere si trei API-uri care raspund fiecare altfel.